CSC3206 ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1

NO.	MEMBER	STUDENT ID	CONTRIBUTION
1	Liang Yrenn Da	22067128	A* Search
2	Ooi Jin Yon	22039069	Breadth-First Search
3	Yeoh Jie Long	22071112	Breadth-First Search
4	Kavinesh Mardad	22033369	A* Search
5	Marcus Ong Xun	22019020	Depth-First Search
6	Jimmy Teng Hau Ming	22047179	Depth-First Search

Table of Contents

1.0 Introduction 3

2.0 Problem Definition and Data Collection 4

3.0 Problem Analysis 7

4.0 AI Algorithms 9

4.1 Depth-First Search 9

4.1.1 Evaluation of DFS 12

4.2 Discussion for DFS performance 13

4.3 Summarize 14

4.2 Breadth-First Search 14

4.2.1 Evaluation of Breadth-First Search 16

4.2.2 Discussion of BFS Performance 17

4.2.3 Summary 18

4.3 A* Search 19

4.3.1 Evaluation of A* Search 21

4.3.2 Discussion of A* Search Performance 22

4.3.4 Summary 23

5.0 Algorithm Comparison 24

5.1 Efficiency 24

5.2 Accuracy of Solution 24

5.3 Real-World Applicability 24

5.4 Problem Constraints 25

5.5 Summarize 25

6.0 Conclusion 25

7.0 Reference 27

8.0 Appendix 28

8.1 Presentation Video 28

8.2 Group Collaboration Evidence 28
8.3 AI Usage Transparency 30
8.4 Reflection 31
8.4.1 Liang Yrenn Da 31
8.4.2 Ooi Jin Yon 32
8.4.3 Yeoh Jie Long 33
8.4.4 Kavinesh Mardad 35
8.4.5 Marcus Ong Xun 36
8.4.6 Jimmy Teng Hau Ming 37

1.0 Introduction

Among the many applications of Artificial Intelligence (AI) in solving real-world problems, route planning is the most common and effective one. Search algorithms are utilised to determine efficient paths when constraints such as travel distance, cost and time are taken into account. Delivery managements in logistics, ride-sharing services and navigation applications regularly apply these functions (Kozerska, 2025). In this assignment, our group analyses a route planning problem that begins at Sunway University and visits every residential location of the group members at least once. The objective of this problem is to determine an optimal route that efficiently reaches the destinations while also minimizing the overall travel costs. For this analysis, the actual navigation data such as the distance, travel time and estimated cost are collected from Google Maps (Google, 2026). The collected data is then used to model the route planning scenario as an AI search problem. This report will evaluate three AI search algorithms; Depth-First Search (DFS), Breadth-First Search (BFS), and A* Search. A comparison of these algorithms will identify the most suitable route planning algorithm based on the efficiency, quality of solution and its practicality in real-world scenarios.

2.0 Problem Definition and Data Collection

Table 1

Residential Location from each member

No.	Member	Location
1	Liang Yrenn Da	Taman Kinrara, Puchong
2	Ooi Jin Yon	Persiaran Tasik Barat, Subang Jaya
3	Yeoh Jie Long	USJ20, Subang Jaya
4	Kavinesh Mardad	Jalan Templer, Petaling Jaya
5	Marcus Ong Xun	Bandar Puteri Puchong, Puchong
6	Jimmy Teng Hau Ming	Jalan Kerinchi, Kuala Lumpur

The route planning scenario in this project begins at Sunway University and link to the residential area from our group members. The objective of this scenario is to determine an efficient route to all residential locations to visit while minimise the travelling distance. The navigation data was collected using Google Maps to obtain travel distance and estimated travel times between each location. **Table 1** shows the residential locations shared by each group member.

Table 2

Distance and Estimated Travel Time from Sunway University to residential locations

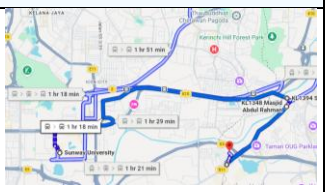
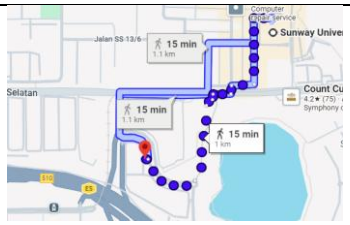
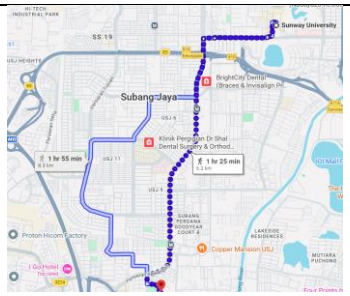
Location	Transportation Method	Distance	Estimated Travel Time
Taman Kinrara, Puchong	**Driving**	**9.2 KM**	13 minutes
	Walking	**8.3 KM**	1 hr 54 minutes
	Public Transport	N/A	1 hr 18 minutes
Persiaran Tasik Barat, Subang Jaya	**Driving**	**1.3 KM**	5 minutes
	Walking	**1 KM**	15 minutes
	Public Transport	N/A	N/A
USJ20, Subang Jaya	**Driving**	**6.2 KM**	13 minutes
	Walking	**6.2 KM**	1 hr 25 minutes
	Public Transport	N/A	48 minutes

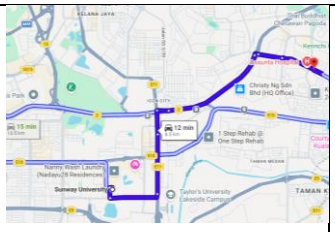
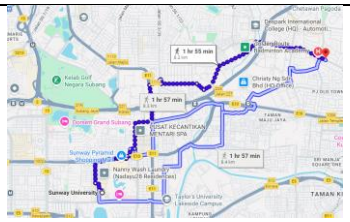
Jalan Templer, Petaling Jaya	**Driving**	**8.5 KM**	12 minutes
	Walking	**8.2 KM**	1 hr 55 minutes
	Public Transport	N/A	1 hr 16 minutes
Bandar Puteri Puchong, Puchong	**Driving**	**11.7 KM**	18 minutes
	Walking	**17.8 KM**	4 hr 5 minutes
	Public Transport	N/A	1 hr 54 minutes
Jalan Kerinchi, Kuala Lumpur	**Driving**	**11.5 KM**	13 minutes
	Walking	**13.6 KM**	3 hr 14 minutes
	Public Transport	N/A	59 minutes

Note: *The table only shows information from Sunway University to each member's residential location*

Table 3

Google Map screenshot

Member's Residence	**Google Map Screenshot**		
	Driving	**Walking**	**Public Transport**
Yrenn Da			
Jin Yon			N/A
Jie Long			

Kavinesh			
Marcus			
Jimmy			

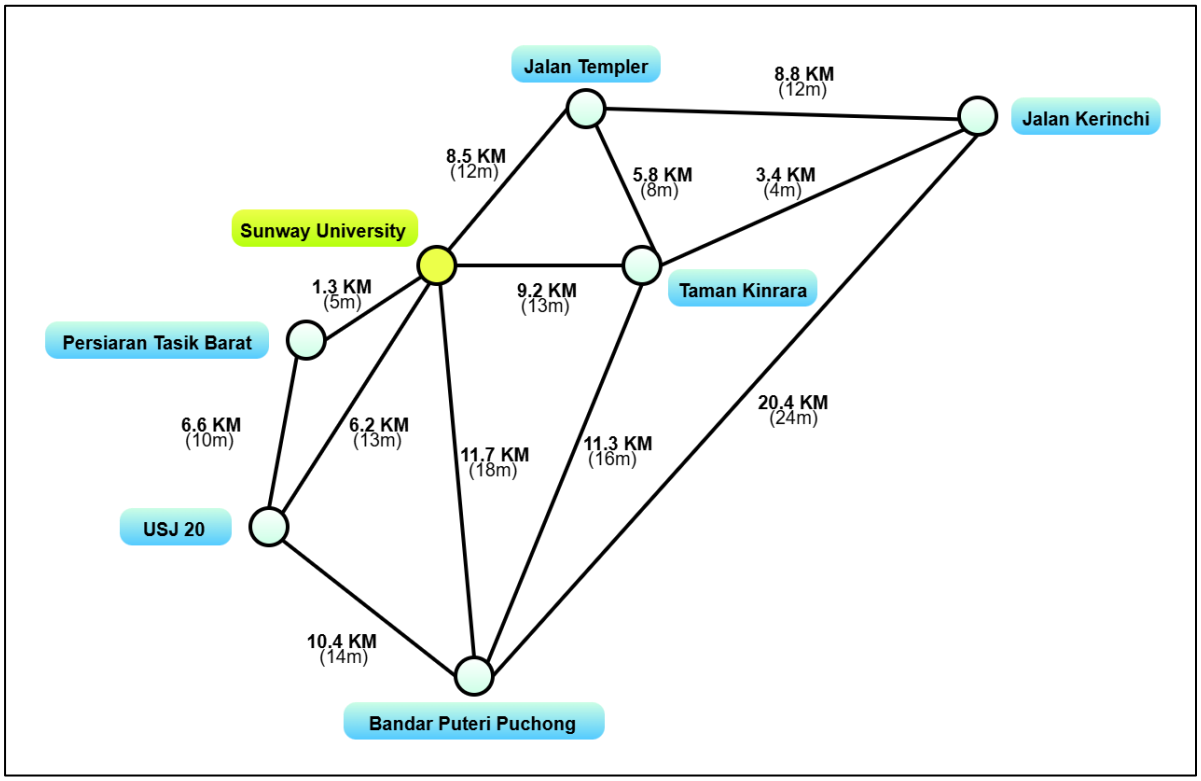
Note: *the table only shows screenshot from Sunway University to each member's residential location*

The information between locations was collected using Google Maps under normal traffic conditions. We had analysed three transportation method which is driving, walking and public transportation. The collected information included distance and estimated travel time. **Table 2** shows the distance and estimated travel time from Sunway University to each residential location. Meanwhile, **table 3** shows the screenshot from Google Map as evidence for the collected data for **Table 2**. The other travelling distances and estimated travel times between residential locations were also collected and represented in **Diagram 1** below.

Based on the collected data, driving transportation provides shortest travel time for most locations compared to walking and public transportation. Walking requires significant longer time than driving and public transportation. Therefore, we will focus on driving transportation for the route planning scenario.

Diagram 1

Weighted Graph Representation of the Route Planning Problem



The weighted graph representation above shows the possible connections between Sunway University and the residential locations of the group members. We also added some connections between each member residential location. The weights assigned to each edge represent the travelling distances and estimated travel times collected from Google Maps under normal traffic conditions. This graph structure allows AI search algorithms to efficiently explore and evaluate different possible travelling routes.

3.0 Problem Analysis

The route planning problem can be formulated as an AI search problem by defining the state space, initial state, goal state, actions, transition model and path cost function. This enables AI search algorithms to systematically explore possible routes and identify efficient solutions.

1) State Space

The state space is a collective of sets of states that one can change to based on all the possible locations as stated in the weighted graph representation above where are

{Sunway University, Jalan Templer, Jalan Kerinchi, Taman Kinrara, Persiaran Tasik Barat, USJ 20, Bandar Puteri Puchong}.

2) **Initial State**

The initial state is the state for all AI search algorithms to start in which will be at the designated root node of Sunway University

3) **Goal State**

The goal state is the state where all AI search algorithm will try to reach and evaluate i it matches this state and only then terminate the session. The goal state for this AI search problem is to reach all nodes in the state space at least once.

4) **Actions**

The set of legal moves the search algorithm can make from its current position where it will always choose between connect to the next few neighbouring nodes which has direct connection to its current node. For example, move from Sunway University to Location A, Location B to Location C and Location C to Location D. Each action changes the current state, and it will continue until every location is visited at least once.

5) **Transition Model**

The transition model is about how the system changes from one state to another after the action is performed (Kumar, 2026). When moving from one location to the other, the current location is updated and the visited location will be added to the visited set. As an example, if the current state is Sunway University, moving to Location A will produce a new state where the current location becomes Location A and it will be recorded as visited. This will continue on until the goal state is achieved.

6) **Path Cost Function**

To evaluate the total cost of the travel route, factors such as travel distance, travel time, fuel consumptions and carbon emissions (Cui & Levinson, 2020). For this analysis, the total travel distance and estimated travel time obtained from Google Maps will act as the primary cost functions. The objective of the search algorithms is to identify a route that visits all locations using the most minimal amount of accumulated travel cost.

7) Assumptions

The following assumptions are made for the structure of the problem.

- The journey begins at Sunway University.
- Each residential location of the members is to be visited once.
- The traffic conditions are to be assumed relatively constant during the journey (i.e.: no significant changes in traffic congestion).
- Data retrieved from Google Maps are accurate in terms of the travel costs measures and route plan.
- The vehicle used in the journey has a standard petrol engine with an average fuel efficiency.
- Road elevation, weather conditions, and vehicle factors such as vehicle cargo load are not taken into account.
- Fuel consumption is directly proportional to the travel distance, and carbon emissions are directly proportional to fuel consumption. Therefore, an increase travel distance will result in higher fuel consumption and carbon emissions.

4.0 AI Algorithms

4.1 Depth-First Search

Depth-First Search (DFS) is a type of uninformed search which is a type of search that does not have any knowledge prior to reaching the goal and work in the blind. It is an algorithm that traverse a graph by always going as far as possible along one path before backtracking. It also follows and uses the stack Last-In, Last-Out (LIFO) discipline when choosing the next edge to explore (Kozen, 1992).

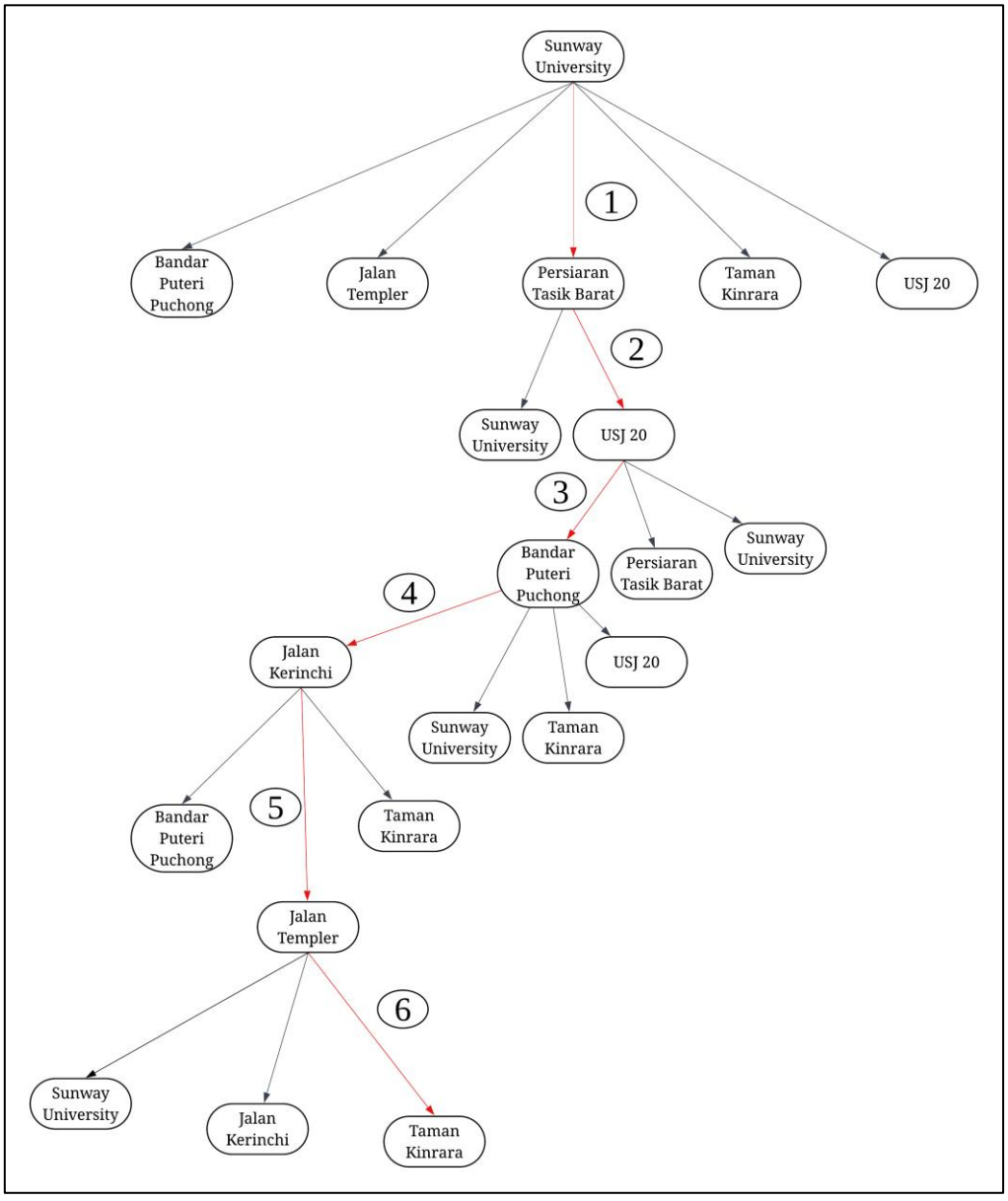
Assumption:

The algorithm will push the neighbouring locations into the LIFO data stack in reverse alphabetical order. Hence the location name with ascending alphabetical order will be expanded first from left to right on the search tree diagram.

A graph search method is also used instead of tree search which helps to removes node location that has already been travelled to as to prevent looping.

Diagram 2

DFS completed diagram journey



Before step 1, starting from the root level (Sunway University), based on the assumptions stated earlier of the algorithm and on the graph representation of the map, the 5 direct connections to the root node are pushed into the LIFO data stacked in reverse alphabetical order and then expanded out from left to right in alphabetical order which is shown in the first level of the nodes directly after the root node. Because of this, the algorithm will explore starting from

Bandar Puteri Puchong node, and if failed to reach the goal it will backtrack and continue towards the next neighbouring node at level 1 downwards. The diagram above effectively shows the first path exploration which has achieved all its goal. Also looping or turning back to previously reached location is prevented as graph search is used.

Step 1: Sunway University to Persiaran Tasik Barat

The algorithm continues its deep dive exploration after the previous two nodes from Bandar Puteri Puchong and Jalan Templer did not reach the goal.

Step 2: Persiaran Tasik Barat to USJ 20

As mentioned earlier due to graph search, going back to Sunway University which we started at is prevented, hence it will travel to USJ 20.

Step 3: USJ 20 to Bandar Puteri Puchong

Since Sunway University and Persiaran Tasik Barat has been explored, the algorithm will choose to travel down the node of Bandar Puteri Puchong.

Step 4: Bandar Puteri Puchong to Jalan Kerinchi

At Bandar Puteri Puchong, the nodes of Jalan Kerinchi , Sunway university, Taman Kinrara and USJ 20 are arranged first and expanded out in alphabetical order, after which the algorithm will check all 4 nodes to see if they have been explored which leaves Jalan Kerinchi and Taman Kinrara where Jalan Kerinchi is chosen based on the alphabetical order.

Step 5: Jalan Kerinchi to Jalan Templer

3 nodes are traversable from Jalan Kerinchi which are Bandar Puteri Puchong, Jalan Templer and Taman Kinrara . Since Bandar Puteri Puchong has been explored and Taman Kinrara is after Jalan Templer in alphabetical order, Jalan Templer is chosen

Step 6: Jalan Templer to Taman Kinrara

From Jalan Templer, Sunway University and Jalan Kerinchi has been explored before hence that leaves the only node option of Taman Kinrara which also subsequently finish the goal of starting from Sunway University as the root node and exploring all possible different location nodes at least once as the goal.

Hence the final ordered path sequence is [Sunway University, Persiaran Tasik Barat, USJ 20, Bandar Puteri Puchong, Jalan Kerinchi, Jalan Templer, Taman Kinrara] after the algorithm confirm the goal has been reached in its evaluation check. DFS algorithm will hence terminate.

4.1.1 Evaluation of DFS

4.1.1.1 Completeness

Depth-First Search is complete for the route planning issue, as the graph search is used to avoid revisiting the previously explored locations. Since the graph consists of a restricted number of locations, DFS will explore all the reachable nodes and find a solution if there is one available. In our case, DFS successfully visited all the residential locations starting from Sunway University and completed the goal state.

4.1.1.2 Optimality

Depth-First Search is not optimal for this case because it does not consider the travel distance or time when choosing the next node. DFS follows the deepest available route according to the predetermined search order. Therefore, the route created by DFS has a total travelling distance of 53.3km and an estimated travelling time of 73 minutes. Although the route achieves the goal of visiting all the locations, there is no guarantee that the route is the shortest or lowest cost.

4.1.1.3 Time complexity

The formula for the time complexity of Depth-First Search is $O(b^m)$, where b is the branching factor and m is the maximum depth of search tree. In the worst case, DFS may explore to many paths before reaching the objective state. However, the search process can be achieved within a reasonable time because the route planning graph only consists of seven locations.

4.1.1.4 Space complexity

The formula for the space complexity of Depth-First Search is O(bm). Depth-First Search only stores the current path and a restricted number of unexplored nodes in memory. As a result, the memory requirement is low compared to algorithms that store all generated nodes at each level. This makes DFS suitable for the problem where the efficiency of the memory is crucial.

4.1.1.5 Practical feasibility

Depth-First Search is practical for small route planning problems because it requires little memory, and it is simple to apply. In this scenario, DFS created a complete route that covers all the locations with a total travelling distance of 53.3km and a total travelling time of 73 minutes. However, the result route may not be efficient for a real-world navigation system where minimising the travel distance and time is the major goal, because the algorithm does not consider route costs during node selection

4.2 Discussion for DFS performance

Depth-First Search (DFS) is useful for this route planning problem, as it is simple to execute and requires little memory. In this case, Depth-First Search successfully visited all the residential locations and created a complete route from Sunway University. The graph search approach also avoided visiting the previous explored locations again, preventing unnecessary loops.

However, DFS does not consider travelling time or distance when picking the next node. Instead, DFS follows the deepest accessible path according to the predefined search sequence. Therefore, the route covered a travel distance of 53.3km and a travel time of 73 minutes, even though there might be a more efficient route that exists. As a result, DFS is able to find a valid solution, but the lowest travelling cost is not guaranteed.

Compared to A* search, DFS is more efficient in terms of memory, but route optimisation is less effective, as DFS ignores edge weights during the node search process. While DFS is suitable for graph exploration and traversal, it is not useful for a real-world route planning, where reducing the travel time and distance is crucial

4.3 Summarize

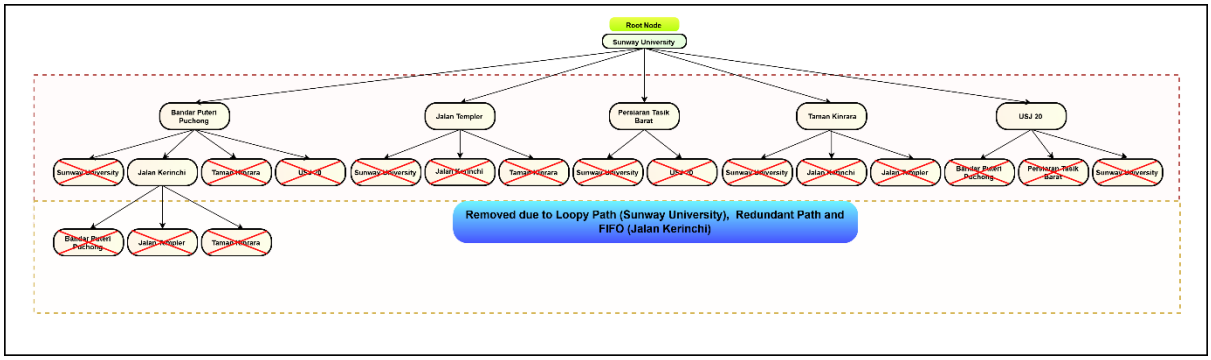
To summarize this, Depth-First Search have completely explored all the residential locations and created a complete route for the route planning problem. This algorithm showed the efficiency of memory and simple to execute, while making sure that all locations were visited. However, DFS does not consider travelling time and distance, or other route costs when picking the nodes for expansion. Therefore, although DFS can find a valid solution, the most efficient route is not guaranteed. As a result, DFS is suitable for graph exploration and traversal tasks, but it is not useful for real-world route optimisation.

4.2 Breadth-First Search

Breadth-First Search (BFS) is an uninformed search algorithm that starts from a source node and explores the graph level by level that used to find the shortest path in an unweighted graph (Beamer et al., 2013; Silvela & Portillo, 2001). It uses a First-In-First-Out (FIFO) queue data structure to manage the searching process, where the first inserted node will be explored and removed. The FIFO queue can be implemented using an array with head and tail pointers, where enqueueing inserts an item at the tail of the queue while dequeueing removes an item from the head of the queue (Leiserson & Schardl, 2010).

Diagram 3

BFS tree diagram



Before Step 1, starting from the root level (Sunway University), based on the BFS algorithm principles and the map's graph representation. The root node is expanded first before exploring its neighbouring residential locations level by level using a First-In-First-Out (FIFO) queue.

Step 1: Expand the Root Node (Level 0)

Five residential location is directly connected to Sunway University are generated, which are Bandar Puteri Puchong, Jalan Templer, Persiaran Tasik Barat, Taman Kinrara, and USJ20. These nodes are added based on the FIFO order.

Step 2: Level 1 Nodes

Since BFS expand the node level by level, the algorithm start travel to the neighbouring residential locations directly connected to Sunway University before proceeding to the next level, which is Bandar Puteri Puchong, Jalan Templer, Persiaran Tasik Barat, Taman Kinrara and USJ20 in FIFO order.

Step 3: Level 2 Nodes

After all Level 1 nodes have been expanded, it proceeds to Level 2 nodes.

It began to develop Bandar Puteri Puchong and produces Sunway University, Jalan Kerinchi, Taman Kinrara, and USJ20 as its child node. Sunway University is eliminated due to the loop in the path. Meanwhile, Taman Kinrara and USJ20 are removed because they already exist in the frontier. Therefore, only Jalan Kerinchi is retained.

After that, Jalan Templer is expanded and generates Sunway University, Jalan Kerinchi, and Taman Kinrara. Sunway University and Taman Kinrara are excluded for the same reasons outlined above. Jalan Kerinchi is also removed because it was already generated earlier under Bandar Puteri Puchong node. The FIFO principle states that the first occurrence of Jalan Kerinchi is kept, and the later occurrences are discarded.

When Persiaran Tasik Barat, Taman Kinrara, and USJ20 are expanded, their child nodes are removed because they are either creating loopy paths or redundant paths. Therefore, no additional nodes are added.

Step 4: Level 3 (Termination)

When it continues to expand Jalan Kerinchi, the algorithm removed its child node. Therefore, the BFS search process is terminates.

Although BFS successfully discovered all residential location in the graph, but it failed to produce a single continuous travelling route that visits all residential locations from Sunway University. This happened when these 5 locations are closely linked together.

4.2.1 Evaluation of Breadth-First Search

4.2.1.1 Completeness

Breadth-First Search (BFS) is a complete search algorithm for this route planning problem because it systematically visits all the nodes. By using a graph search approach, previously visited locations are not revisited, preventing infinite loops and redundant exploration. As the route planning graph has a finite number of houses and connections, if there is a solution then BFS will find it eventually. In this scenario, BFS was able to identify all 6 residential locations starting from Sunway University. Thus, BFS is a complete search algorithm and can always find a solution if it exists in the search space.

4.2.1.2 Optimality

Breadth-First Search is best used for a graph where all the edges have the same weight, or cost. This route planning problem has a graph representation of the locations for houses and the distances and travelling times between them. The BFS algorithm simply grows nodes by their depth, it doesn't take into account the weight of each edge while performing the search. Therefore, the first solution discovered by BFS may not be the shortest path, nor the smallest travelling time. As a result, BFS is not optimal in this weighted route planning problem and is considered non-optimal in this regard.

4.2.1.3 Time Complexity

The time complexity of Breadth-First Search is O(b^d), where b is the branching factor and d is the shallowest solution's depth. In BFS traversal, all nodes at the same level are explored from left to right and before going to the next level. Thus, the number of nodes created grows exponentially with the depth of the search tree. The number of locations on the graph is small

enough for the route planning used for this assignment that the computational cost is manageable. However, for larger route planning networks containing hundreds or thousands of locations, the execution time would increase significantly due to the large number of nodes that must be explored.

4.2.1.4 Space Complexity

The space complexity of Breadth-First Search is O(b^d). In contrast to DFS, which only keeps the current search path, BFS will require maintaining a frontier of all nodes generated at each level in the search path before they can be expanded. This means more memory will be used up because a large number of nodes will exist in the queue at the same time. In this assignment, memory requirement is sufficient because the graph size is small, but for larger-scale transportation networks with limited memory resources, BFS may not be practical.

4.2.1.5 Practical Feasibility

Breadth-First Search is useful for traversing graphs and finding all possible destinations. The algorithm is straightforward to implement and guarantees that a solution will be found if one exists. In this assignment, BFS was able to explore all the residential locations connected to Sunway University and showed a systematic exploration strategy. However, BFS does not take travelling distance, travelling time, fuel consumption, or traffic conditions into consideration during node expansion. As a result, the algorithm can be used for graph exploration but it is less appropriate for real-world route optimisation problems in which minimisation of travel cost is among the key objectives.

4.2.2 Discussion of BFS Performance

Breadth-First Search has several advantages for the route planning problem. The algorithm is complete and it systematically explores all reachable places, one level at a time. This makes BFS appropriate when it is important to know each and every location is discovered. Moreover, the algorithm is not a very complicated one and it is easy to understand in comparison with other search algorithms.

However, BFS also haveseveral limitations. The algorithm needs a lot of memory since the frontier nodes need to be saved throughout the search process. Moreover, BFS ignores the cost

of the edges, which are, for instance, travelling distance or travelling time. Therefore, algorithm may find a solution that is valid but not the most efficient travelling route.

BFS is a systematic exploration method compared to Depth-First Search (DFS) because it first explores the neighboring locations and then explores deeper levels in the graph. This minimizes the chances of taking too long of a branch and thinking about other options. In addition, BFS ensures that the shallowest solution is discovered first, while DFS can take many paths to find a solution.

However, DFS needs much less memory than BFS, since all nodes on the current frontier need to be stored in memory. However, DFS has smaller memory requirements than the other because it only saves the current search path.

When compared with A* Search, BFS performs less effectively for route planning problems. A* Search uses heuristic information via the evaluation function $f(n) = g(n) + h(n)$ to maximize the selection of more promising moves and minimize unnecessary searches. On the other hand, BFS is a depth-first search with no heuristic guidance, simply expanding nodes based on depth level. As a result, A* Search can more often than not be able to find more efficient paths (with less travelled cost and smaller travelling time).

In general, BFS can be used for graph exploration and for teaching of search processes. However, for practical route planning applications where route optimisation is important, A* Search provides a more efficient and realistic solution.

4.2.3 Summary

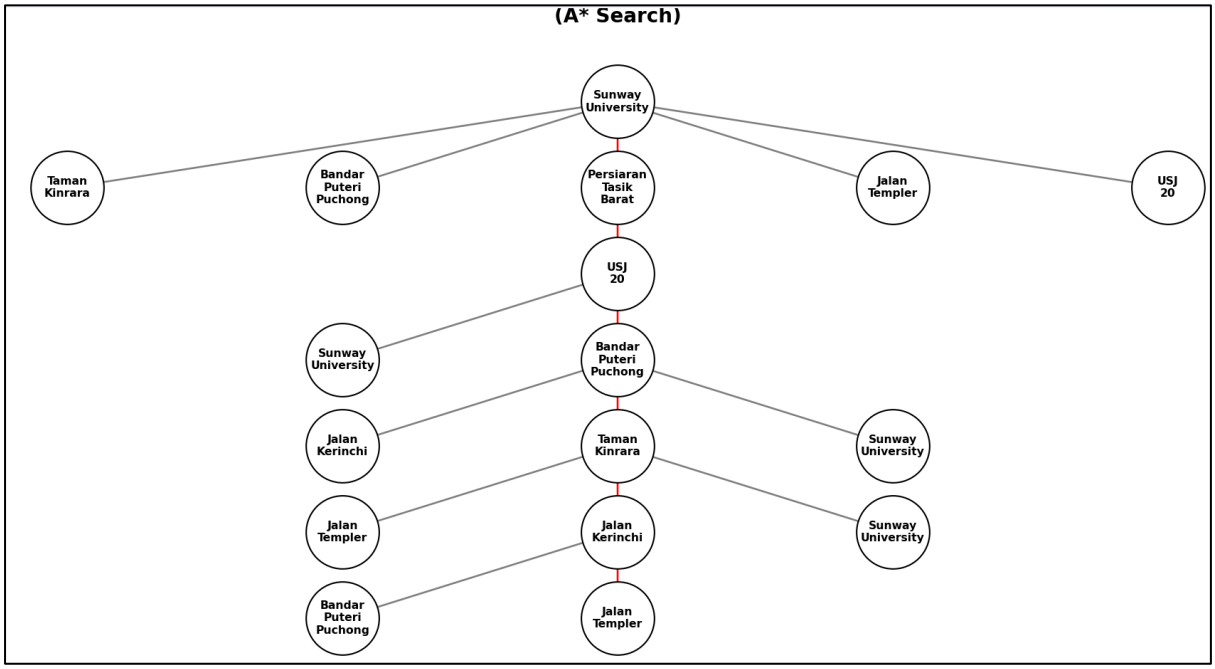
In general, Breadth-First Search has been able to successfully explore all residential areas and successfully showed a complete search strategy for the route planning problem. The algorithm ensures that every reachable location is explored and gives a systematic level by level exploration procedure. However, the BFS does not take into account travelling distance or travelling time or other route costs during node expansion, which means it cannot guarantee the most efficient route. While BFS is useful for traversing graphs and exploring problems, it is not as well suited for real-world applications of route optimisation when compared to heuristic based algorithms like A* Search.

4.3 A* Search

A* Search is an informed search method that efficiently finds the path between a start state and a goal state in a weighted graph by using heuristics to inform the search (Hart et al., 1968; Russell & Norvig, 2020). Instead of uninformed search method like depth-first or breadth-first search, A* Search uses a total cost function, f(n) = g(n) + h(n) where g(n) is the actual cost of getting to node n from the start state, and h(n) is an estimate of the remaining cost from node n to the goal state, gained from knowledge of the problem (Russell & Norvig, 2020). The algorithm implements its frontier using a priority queue data structure that is implemented as an actual binary heap in which nodes are dynamically sorted, and the node with the lowest f(n) is continually popped for expansion (Cormen et al., 2009).

Diagram 4

A* Search diagram



The A* search algorithm starts at the root node which is Sunway University, it expands in accordance with the principles of the A* search algorithm, the graph representation of the map, and a Priority Queue. All direct connections are pushed into this queue and then they are sorted based on the lowest total estimate cost according to the function f(n) = g(n) + h(n), where g(n) is the actual distance travel and estimated travel time and h(n) is the heuristic estimate of the remaining path. Using this function the algorithm will always take the shortest possible path overall in mathematical terms. This helps to remove unnecessary loop or revisiting a node

without any gain.

Step 1: Sunway University to Persiaran Tasik Barat

The algorithm starts from root node and checks all direct connections. Since Persiaran Tasik Barat has the smallest actual travel distance of 1.3 km (5 minutes). This gives the lowest evaluation score for all of the f(n) functions in the priority queue so it is expanded first.

Step 2: Persiaran Tasik Barat to USJ 20

After reaching to Persiaran Tasik Barat from Sunway, it is penalized in the queue since it does not decrease the number of remaining target locations. The algorithm instead will explore to USJ 20, which take 6.6 km (10 minutes) as its f(n) is the best possible total cost to clear the next unexplored node.

Step 3: USJ 20 to Bandar Puteri Puchong

After explored to Sunway University and Persiaran Tasik Barat, the algorithm will review all the frontier nodes and decide on the next lowest estimate cost to explore, in this case at Bandar Puteri Puchong, covering 10.4 km (14 minutes).

Step 4: Bandar Puteri Puchong to Taman Kinrara

At Bandar Puteri Puchong, the algorithm continues to calculate the actual path costs of the unvisited locations. Where Taman Kinrara is selected to continue as it has the lowest distance of 11.3 km (16 minutes) while Jalan Kerinchi has 20.4 km (24 minutes), making it the first in queue to be selected.

Step 5: Taman Kinrara to Jalan Kerinchi

From Taman Kinrara, the algorithm then checks the nodes that have not been visited. Jalan Kerinchi is closer with a distance of 3.4 km (4 minutes) instead of Jalan Templer of 5.8 km (8 minutes). There this is selected as the next connection.

Step 6: Jalan Kerinchi to Jalan Templer

At this stage, there are 3 nodes that can be reached from Jalan Kerinchi, but since Bandar Puteri Puchong and Taman Kinrara have already been visited. The only node that remains unvisited is Jalan Templer taking 8.8 km (12 minutes), thus the algorithm extends towards it to meet the requirement of visiting every group member's home location at least once.

The final optimal tour sequence yields a total travel distance of 41.8 km with an estimated driving time of 61 minutes.

4.3.1 Evaluation of A* Search

4.3.1.1 Completeness

A* is a complete search algorithm for this route planning problem and it always finds a solution when one exists, as long as the branching factor is finite and the cost of each action is always greater than a very small positive number. It uses a priority queue and a closed list such as graph search approach to systematically process the frontier and avoid infinite loops or data duplication. In this case, A* was able to successfully navigate the network and find a complete tour of all the 6 residential locations from Sunway University, it was a complete search algorithm that would always find a complete path in the search space.

4.3.1.2 Optimality

A* is an optimal algorithm that is well-suited in a weighted graph in where there are edges correspond to different distances or different travel times. It evaluates nodes based on the total estimated cost with the function f(n) = g(n) + h(n). Moreover, as cost of travelling to the goal is tracked precisely and h(n) is designed to be never overestimate the actual remaining cost to the goal, A* is mathematically proven to find the path with the lowest cost. As a result, A* is optimal in this route-planning problem with weights to find the most efficient route.

4.3.1.3 Time Complexity

The time complexity of A* Search depends on the quality of the heuristic function used, the worst-case scenario can be exponential O(b^d). Since A* Search uses the heuristic to direct the search straight to the goal, the time required to compute the route planning for this assignment

is very small. Although it is slow for larger route planning networks, A* can still perform very much better than the uninformed searches if a good heuristic is used. In our practical implementation, the time needed to calculate the optimal tour was almost negligee with less than 0.1 seconds. However, this rapid speed execution is primarily due to the very low state space of the assignment's graph. The actual exponential time complexity of A* would be evident when implemented in a real-world large-scale city grid, which is the reason why a good heuristic is essential.

4.3.1.4 Space Complexity

A* Search is that it tends to have space complexity of O(b^d), it is one of the flaw of using this algorithm. A* needs to store all generated nodes in memory in a priority queue and also be able to keep track of all explored nodes to avoid looping. Since it explores multiple paths simultaneously, the amount of memory needed is exponential. As the assignment have small number of residential locations, the memory require is still manageable. However, A* can be impractical for large-scale transportation systems with massive number of nodes due to limited memory resources.

4.3.1.5 Practical Feasibility

A* Search is very practical and is standard for real world routing and navigation systems. The algorithm considers travelling distance, travelling time, and variable edge cost in each step of node expansion. It was in this assignment that A* was able to make intelligent decisions about which residential locations to navigate to, given that these were linked to Sunway University and were prioritized by the shortest logical jump. A* is very suitable and useful for real-world route optimisation problems, as it is designed to reduce the travel cost as its sole goal.

4.3.2 Discussion of A* Search Performance

There are a several advantages of A* Search in terms of the route planning problem. The algorithm is both complete and optimal, ensure that all reachable places will be visited and guarantee that the path chosen will be the most cost-effective path possible. This allows A* Search to be highly appropriate when the objective is to minimise travel distance, fuel usage or travelling time. Using heuristic guidance makes it more intelligent than blind search methods.

However, there are also some drawbacks to A*. This algorithm will use up a lot of memory because all of the nodes created on the frontier need to be stored throughout the search to determine which nodes have the lowest f(n) value. Moreover, A* can only work correctly if its heuristic function h(n) works accurately. If the heuristic used is poor or non-admissible, the algorithm might lose its optimality or reduced execution time.

A* is a much better search to use in this particular case than Breadth-First Search (BFS) which is a blind search that does not take into account the edge costs, but the depth level of the nodes. This results in BFS finding a valid but highly inefficient travelling route. A* is able to do this by computing the cost of the path that it takes, and discards the shallowest solution if a cheaper, but deeper, solution is found.

Meanwhile, A* utilize much more memory than Depth-First Search (DFS) which requires little memory since only the current search path is saved. However, the cost in DFS is not optimum as it can easily select the longest and the least efficient one, based on the order of the nodes expanded. While A* compromises on the memory usage and ensures the optimal route.

In general, BFS and DFS can be employed for the basic exploration of a graph but have limitations when edge weights are involved. Where route optimisation is necessary, A* Search is a much more efficient, realistic and mathematically sound solution for route planning applications.

4.3.4 Summary

Overall, A* able to successfully navigate through all residential areas and showed a very efficient and optimal search strategy in the route planning problem. The algorithm guarantees that the minimum travelling distance and time are minimized during node expansion and that the algorithm explores every reachable location. It uses the evaluation function f(n) = g(n) + h(n) to ensure the most efficient route is discovered. A* Search uses more memory to store its search frontier, but the algorithm is ideally suited to real-world applications such as route optimisation and is much more efficient than uninformed search algorithms in a transportation network environment.

5.0 Algorithm Comparison

5.1 Efficiency

Depth-first search (DFS) is computationally efficient compared to the others, in terms of the memory usage, as DFS only stores the current path and a restricted number of unexplored nodes. But, DFS may explore those unnecessary routes before reaching a solution. Breadth-first search (BFS) will systematically explore all the nodes by level. However, it requires more memory, as all the frontier nodes must be stored in the queue. A* search is the most efficient algorithm for this route planning problem because it applies heuristic data and actual path costs to lead the search toward promising routes, and it reduces unnecessary route exploration.

5.2 Accuracy of Solution

Depth-first search completely created a route, but it ignored the travelling time and distance during the node selection. The total distance travelled is 53.3km, and it takes 73 minutes in total. Breadth-first search successfully explored all the residential locations, but failed to generate a continuous travelling route that can visit all locations in sequence. However, A* search assesses both the estimated remaining cost and the actual travelling cost using the function $f(n) = g(n) + h(n)$, the result shows that A* Search travel a distance of 41.8 km with an estimated driving time of 61 minutes. Because of this, A* search is able to determine the most cost-effective route and deliver the most accurate solution compared to the other algorithms.

5.3 Real-World Applicability

Depth-first search is applicable for some simple graph traversal problems, but it is less suitable for a practical navigation system, as DFS does not consider any route costs. Breadth-first search is useful for exploring all the reachable locations and connectivity within a graph. However, BFS is not suitable for routes that require optimisation in weighted networks. A* search is broadly used in real-world route optimisation, navigation systems, and logistics planning because it integrates travel distance and cost information when it requires decision making. As a result, A* search best reflects the requirements of real-world route planning applications.

5.4 Problem Constraints

The route planning issue consists of weighted edges representing the travelling distance and the estimated travel time. Since reducing the travel cost is the key objective, algorithms that ignore the edge weights are not suitable. DFS and BFS do not apply the data of travelling distance and time during node expansion, which causes them to generate incomplete or inefficient route planning results. A* search directly integrates the route costs into its decision making process. As a result, A* search is suited well to the optimisation requirements and weighted graph.

5.5 Summarize

By comparing the depth-first search, breadth-first search, and A* search, A* search is the most suitable algorithm for our route planning issue. Though DFS and BFS are able to explore the graph and discover valid solutions, neither algorithm effectively optimises the travelling time and distance. A* search integrates the actual route cost with heuristic guidance to determine the route with the least cost. As a result, A* search delivers a balance between real-world applicability, efficiency, and solution quality for this route planning.

6.0 Conclusion

This assignment successfully demonstrated how Artificial Intelligence (AI) search algorithms can be applied to solve a real-world route planning problem. To achieve this objective, Sunway and the respective residences of all the group members were represented as a weighted graph and three search algorithms (Depth-First Search (DFS), Breadth-First Search (BFS) and A* Search) were analysed and evaluated for the capability of generating an efficient travelling route. The results showed that all three algorithms were able to explore the search space, but with varying degrees of performance in terms of travelling distance, travelling time and route optimisation.

Firstly, Depth-First Search could produce a full route with relatively low memory usage. However, it did not consider route costs during node expansion, which resulted in a route that was not guaranteed to be efficient. Subsequently, Breadth-First Search systematically explored all reachable locations and ensured that all residential areas could be discovered. However, the

algorithm proved to be memory-intensive and failed to provide an optimized continuous traveling route, as it did not consider the weights of the connections. Hence, both DFS and BFS are useful tools for graph exploration and education, but not very practical in the context of weighted route planning problems.

Following up, out of the three algorithms. It was proven that A* Search was identified as the most suitable algorithm for this route planning problem. From the efficiency perspective, A* reduces unnecessary exploration by incorporating heuristic guidance into its decision-making process. Furthermore, in terms of accuracy, A* considers the actual cost of the trip, as well as the estimated cost of the remaining path, done with the function $f(n) = g(n) + h(n)$ to find the best path in terms of cost. Moreover, A* has excellent practical use, as similar algorithms are used in navigation applications like Waze and Google Maps for giving efficient routes. Finally, A* is suitable for solving this assignment due to its direct consideration of weighted factors including travelling distance and travelling time, which are the main criteria of this assignment.

This assignment not only focuses on route optimisation but also underscores the broader impact of Artificial Intelligence in addressing real-world transportation challenges. Specifically, effective routing can help minimize fuel usage, decrease carbon emissions, and enhance the overall efficiency of transportation. Consequently, these benefits align with sustainable urban development initiatives and demonstrate how AI technologies can support smarter decision-making processes in modern cities. Thus, A* Search is the best of technical solutions for this assignment, and it demonstrates the ability of AI to offer practical and sustainable solutions to real-world problems.

In conclusion, while DFS and BFS remain valuable algorithms for graph traversal and fundamental AI learning, A* Search offers the best balance between efficiency, solution accuracy and real-world applicability. Its overall performance in intelligently prioritising promising routes makes it the best algorithm for weighted route planning problems and thus the most suitable for this assignment.

7.0 Reference

Beamer, S., Asanović, K., & Patterson, D. (2013). Direction-Optimizing Breadth-First Search. *Scientific Programming*, *21*(3-4), 702694. https://doi.org/https://doi.org/10.3233/SPR-130370

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. https://www.cs.mcgill.ca/~akroit/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf

Cui, M., & Levinson, D. (2020). Shortest paths, travel costs, and traffic. Environment and Planning B Urban Analytics and City Science, 48(4), 828–844. https://doi.org/10.1177/2399808319897619

Google. (2026). Google Maps. Google. https://maps.google.com

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107. https://ieeexplore.ieee.org/document/4082128/

Kozen, D. C. (1992). *Depth-first and breadth-first search*. In *The design and analysis of algorithms* (pp. 19–24). Springer. https://doi.org/10.1007/978-1-4612-4400-4_4

Kozerska, M. (2025). USING ARTIFICIAL INTELLIGENCE FOR ROUTE OPTIMIZATION AND PLANNING. Scientific Papers of Silesian University of Technology Organization and Management Series11111111, 2025(236), 123–140. https://doi.org/10.29119/1641-3466.2025.236.8

Kumar, R. (2026, January 5). Understanding Search Algorithms in AI: A Comprehensive Guide. Medium. https://medium.com/@reetesh043/understanding-search-algorithms-in-ai-a-comprehensive-guide-860c1d71e9c5

Leiserson, C. E., & Schardl, T. B. (2010). *A work-efficient parallel breadth-first search algorithm (or how to cope with the nondeterminism of reducers)* Proceedings of the twenty-second annual ACM symposium on Parallelism in algorithms and architectures, Thira, Santorini, Greece. https://doi.org/10.1145/1810479.1810534

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. http://lib.ysu.am/disciplines_bk/efdd4d1d4c2087fe1cbe03d9ced67f34.pdf

Silvela, J., & Portillo, J. (2001). Breadth-first search and its application to image processing problems. *IEEE Transactions on Image Processing*, *10*(8), 1194-1199. https://doi.org/10.1109/83.935035

8.0 Appendix

8.1 Presentation Video

https://imailsunwayedu-my.sharepoint.com/:v:/g/personal/22039069_imail_sunway_edu_my/IQAn_WTh-WAHQbsmTGmAWo4mAd-sq4xKTQ_0oFCvsn0N9-U?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=OWb4N8

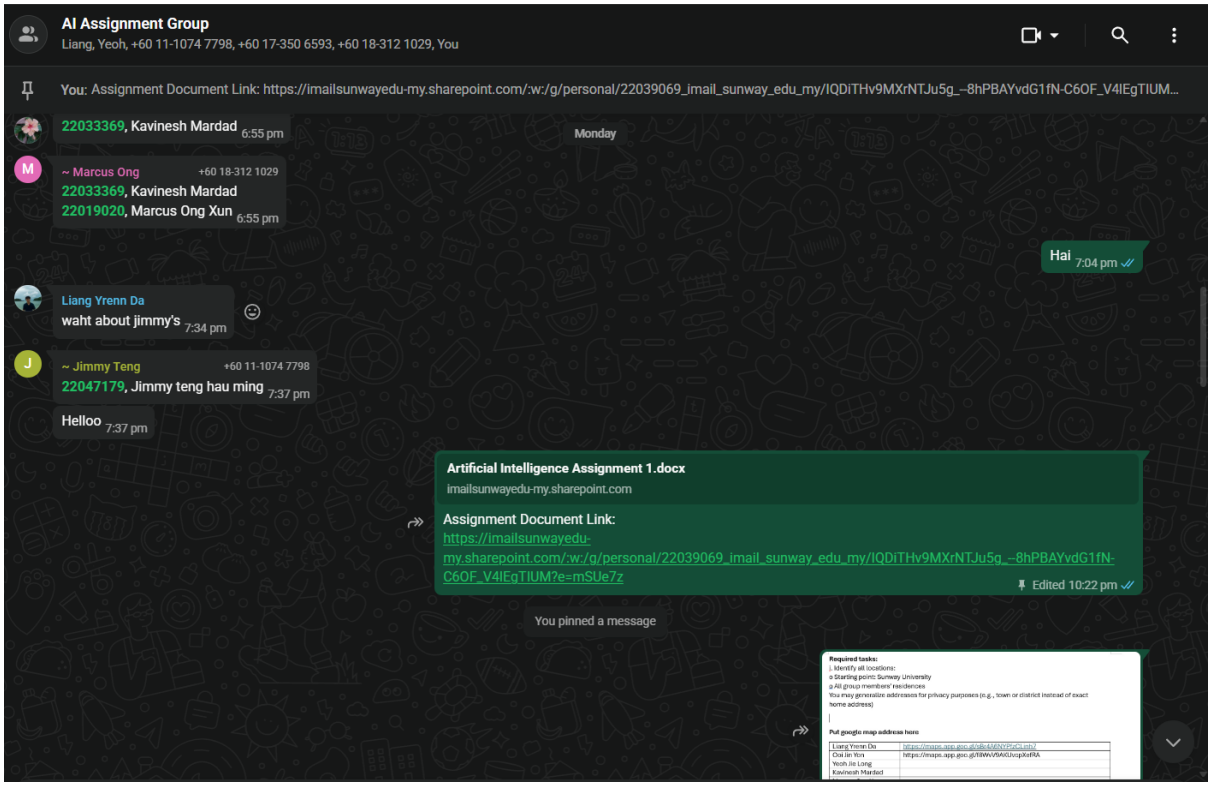
OR

https://youtu.be/J4i14PX84KY

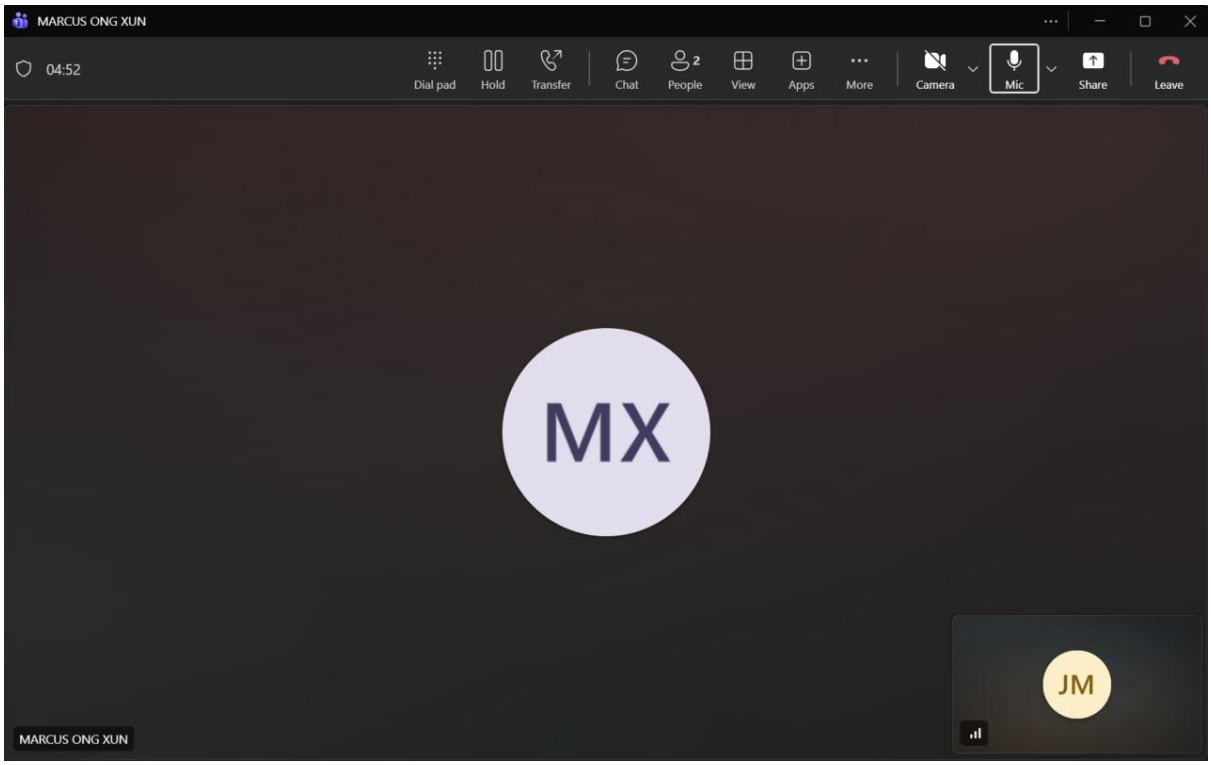
8.2 Group Collaboration Evidence

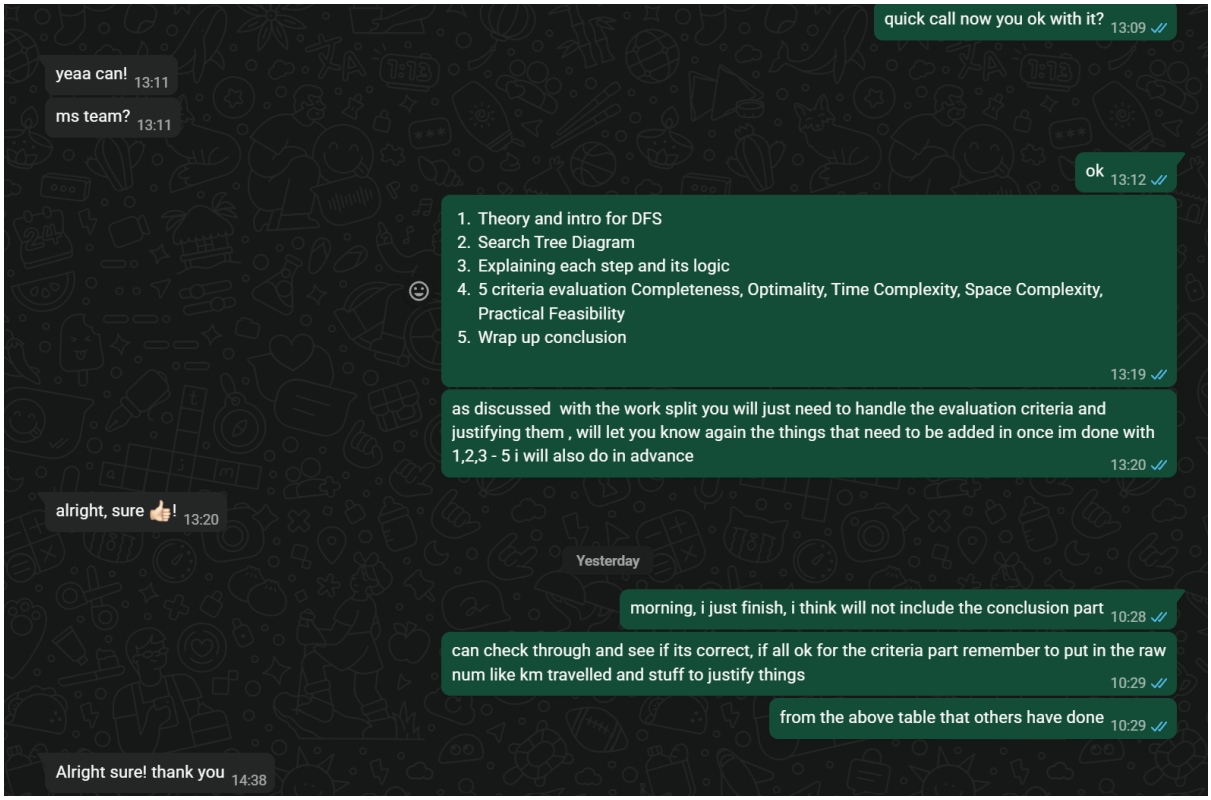
	Section	Person 1	Person 2	Status
1	Introduction	KAVINESH MARDAD	Type to add	Completed
2	Data Collection	OOI JIN YON	Type to add	Completed
3	Problem Analysis	Type to add	Type to add	In progress
4	DFS	KAVINESH MARDAD	LIANG YRENN DA	In progress
5	BFS	YEOH JIE LONG	OOI JIN YON	In progress
6	GREEDY BFS	MARCUS ONG XUN	JIMMY TENG HAU ...	Not started
7	Algorithm Comparison	MARCUS ONG XUN	Type to add	Not started
8	Conclusion	YEOH JIE LONG	Type to add	Not started
9	Evidence of Group Collaboration/AI Transparency	JIMMY TENG HAU ...	Type to add	In progress
10	ndividual Reflection (ALL)	Type to add	Type to add	Not started
11	Presentation Video (ALL)	Type to add	Type to add	Not started

Group work progress tracking



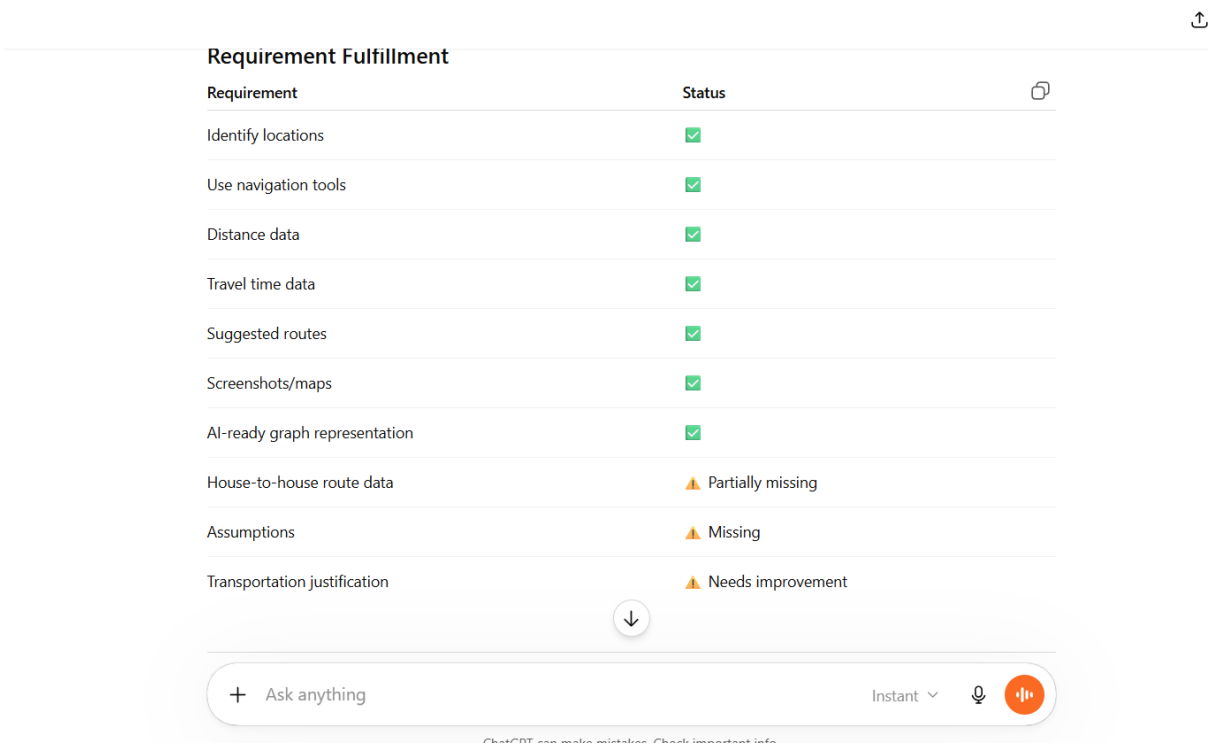
The addition of group members in WhatsApp for group discussions

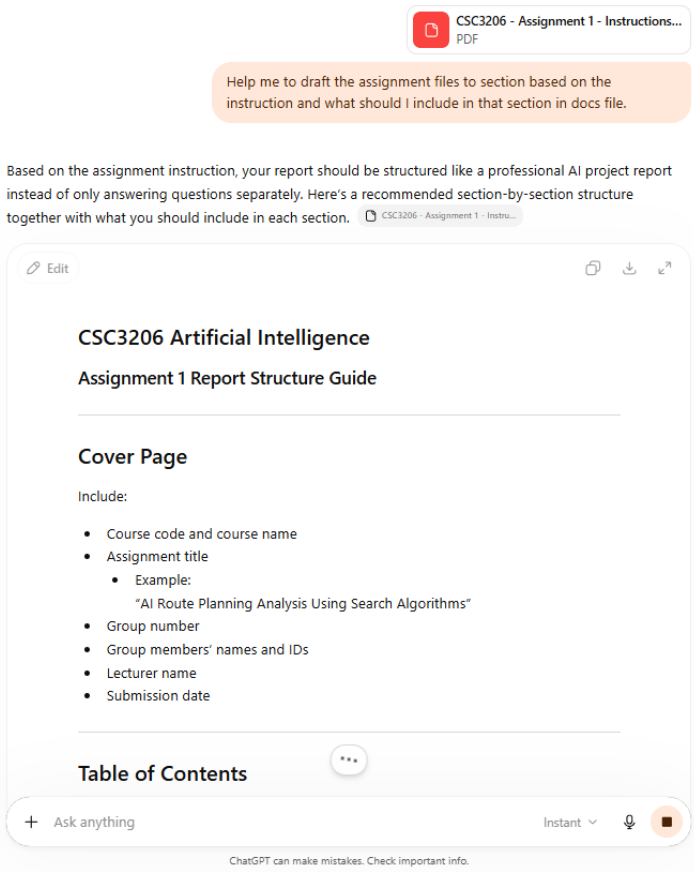




Discussion between Marcus and Jimmy on their assign part of DFS for AI algorithm evaluation

8.3 AI Usage Transparency





8.4 Reflection

8.4.1 Liang Yrenn Da

Before doing this assignment, my understanding of search algorithms was solely in theoretical form, threating this knowledge as a concept and is not fully adapt how this applies in real-world variables such as actually using it to measure distances and travel times. I also didn't notice how uninformed algorithms such as BFS and DFS would have behave with a weighted map network. After completing this assignment, I have gained a concreate knowledge for uninformed and informed algorithm, I no longer view pathfinding as a conceptual flow of nodes. Instead, I understand that finding a valid path isn't solely mathematically based but should also calculate real-word scenario attributes such as different time period and different location and the population.

The most important thing I learned was how to take an algorithm that I had learned in class and apply it to real-world situations. These search methods had not been encountered before in my experience but in a textbook graph with the nodes pre-planned and nice and neat. I was able to model real-world constraints in the environment, such as real-world distances along a real route between Sunway University and the real homes of the members of my team, on a real time to

drive, between these two places. The experience gave me a clear understanding of how a theoretical evaluation function can be directly applied to the real world to create an industry standard navigation tool, from abstract ideas into something that could be used.

The biggest difficulty I had was physically implementing the A* algorithm from scratch. Implementing the conceptual elements like using a priority queue, coding an admissible heuristic function and maintaining a dynamically changing state space in bug-free functional Python code was a significant technical challenge. This hurdle was addressed by employing AI tools strategically to troubleshoot intricate logical issues, streamline data structures, and elucidate code syntax. Along with some focused online research in some technical documentation and developer forums, I was able to find the solution to the gap between theory and execution and was able to build a working application.

Overall, with this assignment I learned more about the concept of AI search algorithm, also, I was able to enhance the theoretical knowledge learnt in the class by implementing the algorithm using a real-world scenario. It also assisted to reinforce my groupwork skill by discussion and collaborative work with my group members when analysing and comparing different search algorithms for the route planning problem.

8.4.2 Ooi Jin Yon

By completing this assignment, I gained a deeper understanding on how AI search algorithms can be used to solve a real-world problem. During the class, I only understood the theoretical concepts of these algorithms. Through this assignment, our group have created a route planning scenario by using our residential location and apply it into DFS, BFS and A* Search algorithm. This implementation helped me gain a better understanding on how different algorithms work to solve the route planning problems.

One of the things that I learned from this is the different algorithms produce different outcomes when solving the same problem like our route planning scenario. The DFS able to find the route without considering the distance or time. Meanwhile, the A* Search able to find

the best route by calculating the cost to find the best solution calculation. However, BFS unable to find a continuous path to all residential location in once as it removes loopy path and redundant path.

One of the challenges encountered when I try to make BFS tree diagrams. I found that the graph has removed a lot of redundant path and loopy path, which cause confusion to me during the expansion process. I have confirmed it again by reading the lecture slide and asking generative AI for clarity. They helped me to have better understanding on the reason it rejected many nodes by explaining the way they handled during the node expansion process. Besides that, I also have checked with my groupmate, Jie Long to ensure the expansion process and node removal were correctly based on the algorithm. Through these discussions, I gained a clearer understanding of how BFS graph search operates and how FIFO ordering affects node expansion and redundant path removal.

Overall, this assignment helped to improve my overall understanding of AI search algorithm by strengthen the theoretical concept learned in class by implement the algorithm using a real-world scenario. It also help to improve my groupwork skill through discussions and collaboration with my group members while analysing and comparing different search algorithms for the route planning problem.

8.4.3 Yeoh Jie Long

My main contribution to this assignment was the Breadth-First Search (BFS) section. My task was to evaluate BFS according to its Completeness, Optimality, Time Complexity, Space Complexity and Practical Feasibility. I also wrote the discussion section in which I compared the performance of BFS with Depth-First Search (DFS) and A* Search. In addition to my own section, I edited portions of the report to make sure it was consistent throughout. For instance, if one of the chosen search algorithms was changed, I was able to see that the introduction was still talking about the previous search algorithm and helped update it to ensure that the report was in line with its final content.

Besides, The most valuable thing I learned from this assignment was understanding how AI search algorithms can be applied to real-world problems. Prior to beginning this assignment, I knew BFS, DFS and A* mostly as a topic I discussed in lectures. I was aware of how they would work in theory, but less familiar with how they might be applied in practical situations. Once I started using real residential addresses, distances to travel, and weighted graphs, I realized how these algorithms could be used in route planning problem. This was helpful when I could use the same vocabulary that I had heard in class for navigation systems like Waze and Google Maps. I discovered that route planning is really a search problem and that depending on the way that the search space is explored, different algorithms can yield very different results.

A difficulty I had was picturing how the algorithms could be linked to a real map in the beginning of the assignment. At first sight BFS, DFS and A* seemed to be just a theoretical problem to solve not on the lecture slides. Once I got the locations as nodes and the travelling distances as edges in a graph, though, it became much clearer to me. I began to grasp the difference in why certain algorithms are more suited for exploration and others for optimisation. It was likely at this stage that the ideas got clearer for me.

During the assignment, I had close cooperation with group members and also checked their work when needed. We discussed the overall direction of the report with each member having a section to discuss, so we could be consistent. I also use ChatGPT as a learning resource when I need to gain insight into something I don't know or if I wanted to see if our report structure matched the requirements from the assignment. Instead of copying and pasting the generated material, I used it to help me formulate ideas, to enhance my understanding, and to determine whether we had addressed the necessary criteria for evaluation.

All in all , if I were given the opportunity to do this assignment again, I would like to be involved in more sections beyond my allocated responsibilities. Due to my schedule and other commitments, I mainly focused on completing my assigned tasks. With more available time, I would like to contribute more towards the overall analysis, assist my teammates in other sections and take a larger role in reviewing the complete report. Overall, this assignment helped

me gain a deeper understanding of AI search algorithms and showed me how theoretical concepts can be applied to solve practical problems in the real world.

8.4.4 Kavinesh Mardad

Throughout the assignment, I contributed towards the introduction, problem analysis and the evaluation of the search algorithms. For the introduction it was essential for me to accurately establish the fact of the real-world applications of AI search algorithms, i.e.: in route planning, and how it is relevant to the scenario of this assignment. The problem analysis is where I had to formulate the scenario into an AI search problem. This included defining the key aspects; state space, initial state, goal state, actions, path cost functions and transition model. Additionally, I contributed in the evaluation of the A* search algorithm.

With this assignment, I was able to understand how the AI concepts I learnt in the lecture sessions can be applied in real-life practical problems. Furthermore, deducing the problem formulation enabled me to value the importance of clearly defining integral components such as the state space and transition model. Doing this assignment also made me realize how A* Search algorithms combine the actual cost and the heuristic estimates to efficiently find the optimal solution. This emphasized for me on why informed search algorithms are often times more suitable than uninformed approaches when in comes to real-world scenarios.

The main challenge I faces was accurately formulating the problem so that it was realistic and feasible. Transforming the route planning scenario into a structured AI model needed proper deduction and reasoning based on research. This was especially true when it came to defining the components such as the transition model and assumptions in the problem. Thankfully, I was able to overcome this hurdle by reviewing the subject materials and also having discussions with my group members to refine the interpretations.

All of us in the group were effective in collaboration, as we divided the tasks based on our personal strengths and we ensured to maintain frequent communication amongst each other. Thus, we were able to stay on path of the assignment requirements and efficiently completed what was needed.

In terms of AI usage, I used AI tools such as ChatGPT to help structure my concepts more effectively and even give suggestions on better words to phrase my explanations.

However, it was imperative that the generated content did not surpass my own input and that any output given by the AI tool was to be double checked and reviewed with relevant online sources (i.e.: research papers and articles). This was to avoid any faults and inaccuracies that can be caused by the AI as it still has the possibilities to make unforced errors.

If I were to redo this project all over again an improvement I would implement is to be more in depth in my problem formulation in the problem analysis section. I would enhance the explanations by adding in visual aids and listing more examples to emphasize my point. Overall, this assignment boosted my understanding of AI search algorithms and improved my ability to apply these techniques in real-world problems.

8.4.5 Marcus Ong Xun

In this assessment, I was responsible for researching and analyzing Depth-First Search (DFS) algorithm and comparing three different algorithms, which are DFS, BFS, and A* Search. In addition, I was also contributing to evaluating the three algorithms based on their problem constraints, efficiency, accuracy, and real-world applicability. I was in charge of preparing the summary and discussion for DFS and the algorithm comparison.

By completing this assessment, I understand how AI search algorithms are able to be implemented to solve the real-world route planning issues. Through this assessment, I realised that each algorithms have their own limitations and strengths. By comparing DFS, BFS, and A* Search, I gained a better understanding of why A* Search is more suitable for weighted graphs, as it considers both heuristic information and the actual costs.

One of the main challenges I faced were to analyze the performance of Depth-First Search and to do some theoretical research in route planning. I also found it challenging to evaluate the algorithms and present the outcome in a clear way. To solve the difficulties, I discussed with my group members and referred back to my lecture notes to make sure that the analysis we made was valid and accurate.

Our group worked well in distributing the tasks fairly and maintaining effective communication

through our WhatsApp group. This is to ensure that all of our members are on the same page and understand their responsibilities clearly.

In addition, I implemented AI tools to brainstorm ideas and clarify concepts. I always verified and compared the information generated by AI with the lecture notes instead of just relying completely on AI-generated information.

If I were to repeat this assignment, I would like to spend more time on analyzing the algorithms and generating a better visualisations to demonstrate the AI search process. I believe this would strengthen the work quality and clarity of the project.

8.4.6 Jimmy Teng Hau Ming

My individual contribution is the creation of assumptions and diagram of DFS journey made using third party tool of lucid chart. I also contributed to the theory introduction of DFS along with the explanation of each step that the algorithm takes and the reasoning behind it. In addition, I am also tasked with contributing majorly for the section of evidence of group collaboration.

I gained a better understanding of AI search algorithms and how they work as I did my own extra research especially for Depth-First Search (DFS) where I gain better clarity of its inner working along with others. I can see and picture better their potential contributions are, advantages and disadvantages of each different search algorithms and their best applications.

Main challenge I faced during assignment is to apply the concept learnt with how we are tasked to do with the assignment which uses our location as node. This requires me to think about how this algorithm will work instead of just understanding it theoretically. For example, I thought DFS is just simple straightforward search algorithm where it continuously searches onwards unless needing to backtrack if goal is not reached without understanding that assumptions are important or else they will be stuck in looping or each iteration of DFS will be

different.

As a group we face problems in understanding at first what kind of AI search algorithms to use and having hypothetical knowledge of which are best. We had group discussions during one of the practical lessons and this helps to be united in our direction and ideas on what to do.

My experience in using AI tools more responsibly and critically is that instead of relying on the answers given, I double check with online sources and research paper to confirm the understanding. I also do not just take the word of their answers as is and would think about it instead of just copy pasting into the document. I also have my fair share of experience where the AI misunderstood what I mentioned and given the wrong answers or directions which would affect the result hence why I would always double check and confirm their replies.

What I would improve is that I should be better prepared theoretically instead of discovering it or understanding things better when I was reading back the lecture slides or when given answers by AI which I am unsure of. This would allow me to have more confidence in my output of work as well as having better direction and vision on how my resulting task should be like.